Experiment 3

Aim :

To create an Employee–Department–Project database schema, insert at least 30 employees across 5 departments and 8 projects, and execute SQL queries demonstrating selection, projection, aggregation, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Description :

This experiment demonstrates the use of SQL for managing and analyzing data stored in an Employee–Department–Project schema. The database contains five departments, eight projects, and at least thirty employees distributed among the departments and projects.

The experiment covers important SQL operations including selection using WHERE, projection using SELECT, aggregation using functions such as COUNT, AVG, SUM, and MAX, grouping records using GROUP BY, filtering grouped results using HAVING, conditional classification using CASE expressions, and sorting query results using ORDER BY.

The schema uses primary keys and foreign keys to maintain relationships between departments, employees, and projects. The queries help generate useful reports such as department-wise employee counts, average salaries, project assignments, salary classifications, and departments satisfying specified conditions.

Source Code :

```
-- Create Department table
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
    location VARCHAR(50)
);

-- Create Project table
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    budget DECIMAL(12,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- Create Employee table
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    salary DECIMAL(10,2),
    hire_date DATE,
    dept_id INT,
    project_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

```
-- Insert 5 departments
INSERT INTO Department VALUES
(1, 'Computer Science', 'Block A'),
(2, 'Human Resources', 'Block B'),
(3, 'Finance', 'Block C'),
(4, 'Marketing', 'Block D'),
(5, 'Operations', 'Block E');

-- Insert 8 projects
INSERT INTO Project VALUES
(101, 'AI Research', 500000.00, 1),
(102, 'Web Development', 350000.00, 1),
(103, 'Recruitment Portal', 180000.00, 2),
(104, 'Financial Analytics', 450000.00, 3),
(105, 'Digital Marketing', 300000.00, 4),
(106, 'Inventory System', 250000.00, 5),
(107, 'Cloud Migration', 600000.00, 1),
(108, 'Business Automation', 400000.00, 5);

-- Insert 30 employees
INSERT INTO Employee VALUES
(1001, 'Aarav Sharma', 'aarav@company.com', 65000, '2022-01-15', 1, 101),
(1002, 'Riya Verma', 'riya@company.com', 72000, '2021-03-20', 1, 102),
(1003, 'Arjun Singh', 'arjun@company.com', 58000, '2023-02-10', 1, 101),
(1004, 'Neha Gupta', 'neha@company.com', 81000, '2020-06-18', 1, 107),
(1005, 'Kabir Mehta', 'kabir@company.com', 69000, '2022-08-12', 1, 102),
(1006, 'Ananya Rao', 'ananya@company.com', 76000, '2021-11-05', 1, 107),

(1007, 'Rahul Kumar', 'rahul@company.com', 48000, '2022-04-11', 2, 103),
(1008, 'Simran Kaur', 'simran@company.com', 52000, '2023-01-25', 2, 103),
(1009, 'Vikram Joshi', 'vikram@company.com', 55000, '2021-09-17', 2, 103),
(1010, 'Pooja Nair', 'pooja@company.com', 61000, '2020-12-01', 2, 103),
(1011, 'Karan Malhotra', 'karan@company.com', 50000, '2022-07-22', 2, 103),
(1012, 'Ishita Jain', 'ishita@company.com', 57000, '2023-05-15', 2, 103),

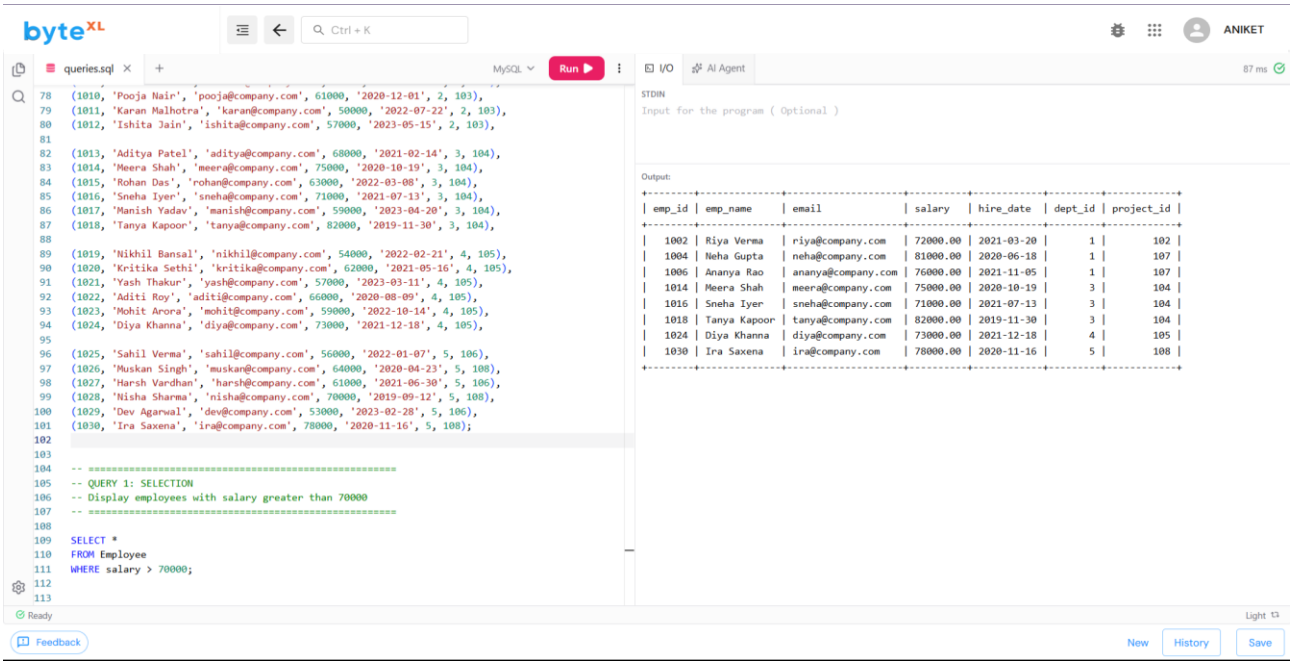
(1013, 'Aditya Patel', 'aditya@company.com', 68000, '2021-02-14', 3, 104),
(1014, 'Meera Shah', 'meera@company.com', 75000, '2020-10-19', 3, 104),
(1015, 'Rohan Das', 'rohan@company.com', 63000, '2022-03-08', 3, 104),
(1016, 'Sneha Iyer', 'sneha@company.com', 71000, '2021-07-13', 3, 104),
(1017, 'Manish Yadav', 'manish@company.com', 59000, '2023-04-20', 3, 104),
(1018, 'Tanya Kapoor', 'tanya@company.com', 82000, '2019-11-30', 3, 104),

(1019, 'Nikhil Bansal', 'nikhil@company.com', 54000, '2022-02-21', 4, 105),
(1020, 'Kritika Sethi', 'kritika@company.com', 62000, '2021-05-16', 4,
105),
(1021, 'Yash Thakur', 'yash@company.com', 57000, '2023-03-11', 4, 105),
(1022, 'Aditi Roy', 'aditi@company.com', 66000, '2020-08-09', 4, 105),
(1023, 'Mohit Arora', 'mohit@company.com', 59000, '2022-10-14', 4, 105),
(1024, 'Diya Khanna', 'diya@company.com', 73000, '2021-12-18', 4, 105),

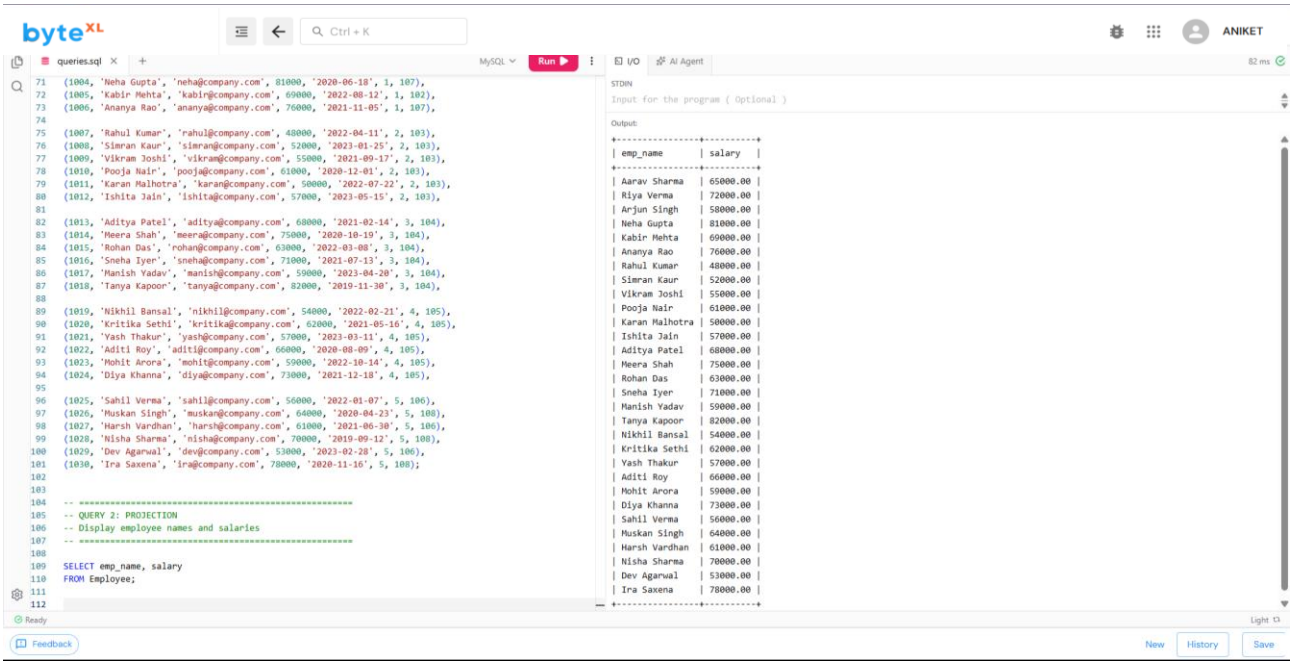
(1025, 'Sahil Verma', 'sahil@company.com', 56000, '2022-01-07', 5, 106),
(1026, 'Muskan Singh', 'muskan@company.com', 64000, '2020-04-23', 5, 108),
(1027, 'Harsh Vardhan', 'harsh@company.com', 61000, '2021-06-30', 5, 106),
(1028, 'Nisha Sharma', 'nisha@company.com', 70000, '2019-09-12', 5, 108),
(1029, 'Dev Agarwal', 'dev@company.com', 53000, '2023-02-28', 5, 106),
(1030, 'Ira Saxena', 'ira@company.com', 78000, '2020-11-16', 5, 108);

-- 1. Selection: display employees with salary greater than 70000
SELECT *
```

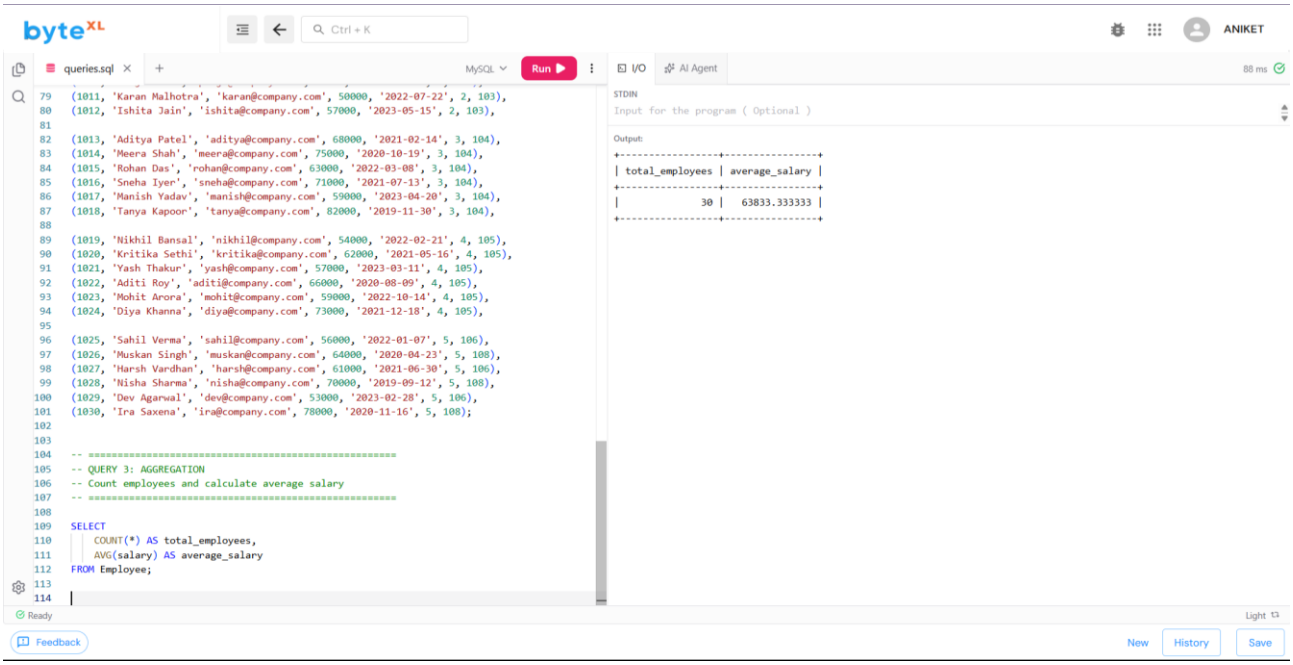
```
FROM Employee
WHERE salary > 70000;
```



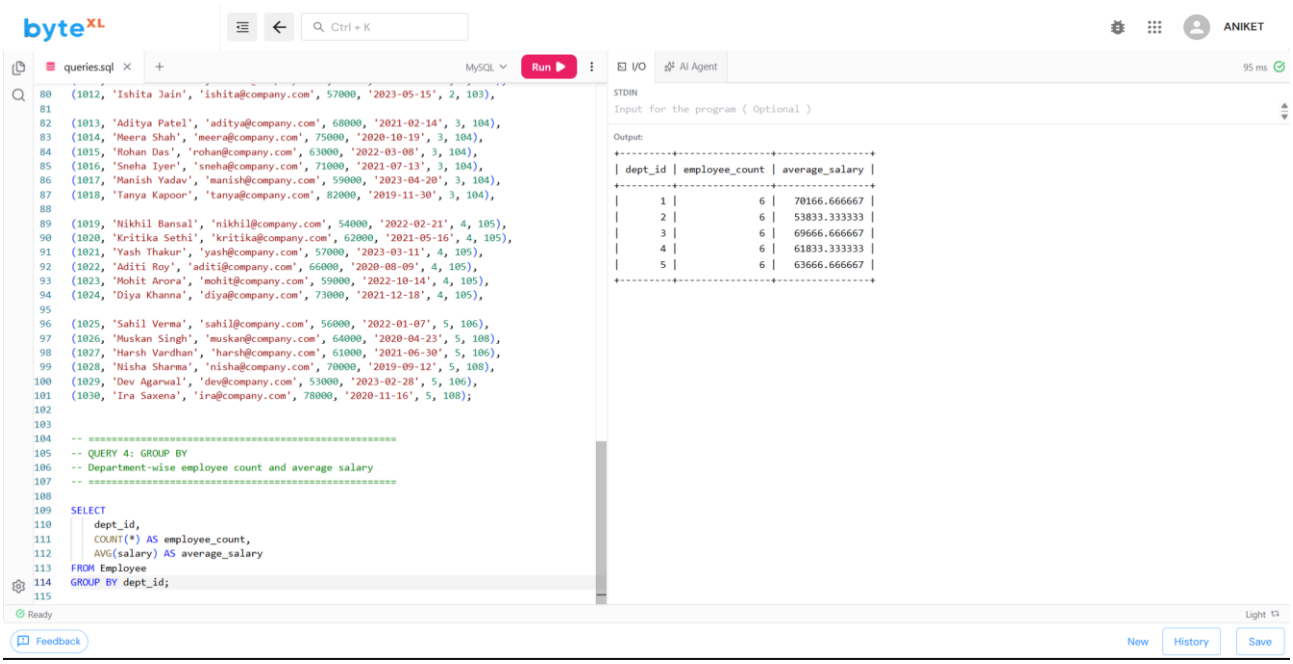

```
-- 2. Projection: display employee names and salaries
SELECT emp_name, salary
FROM Employee;
```



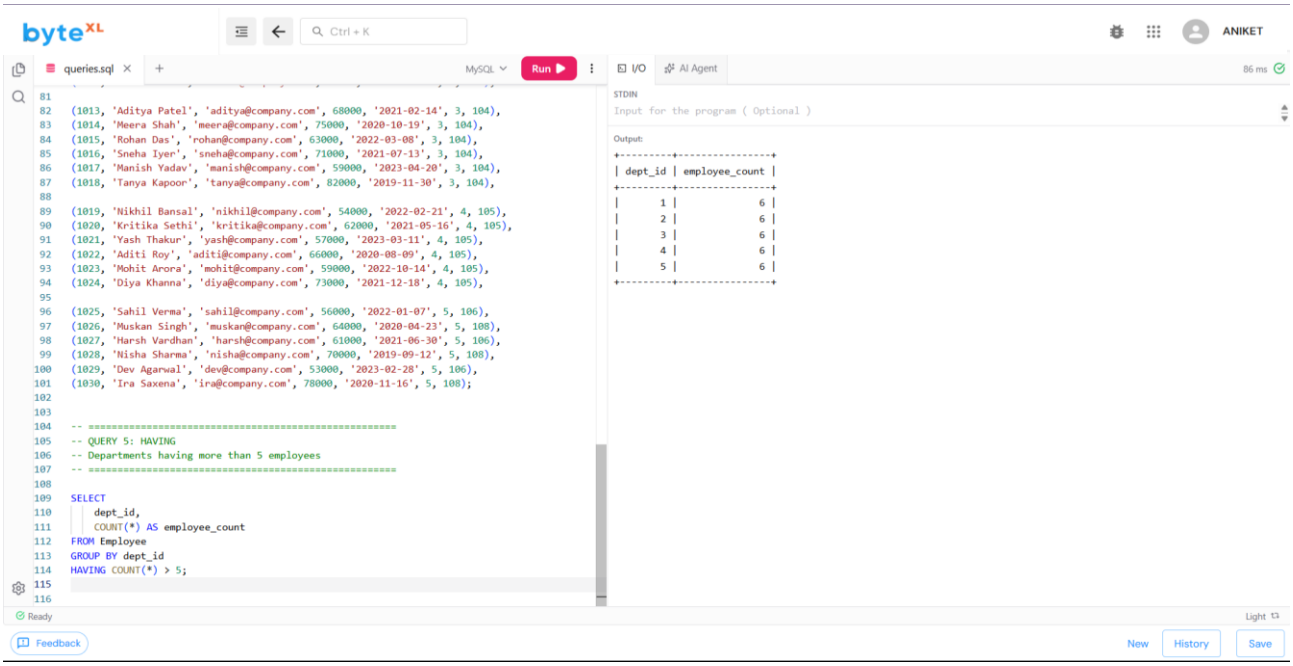

```
-- 3. Aggregation: count employees and calculate average salary
SELECT COUNT(*) AS total_employees,
       AVG(salary) AS average_salary
FROM Employee;
```



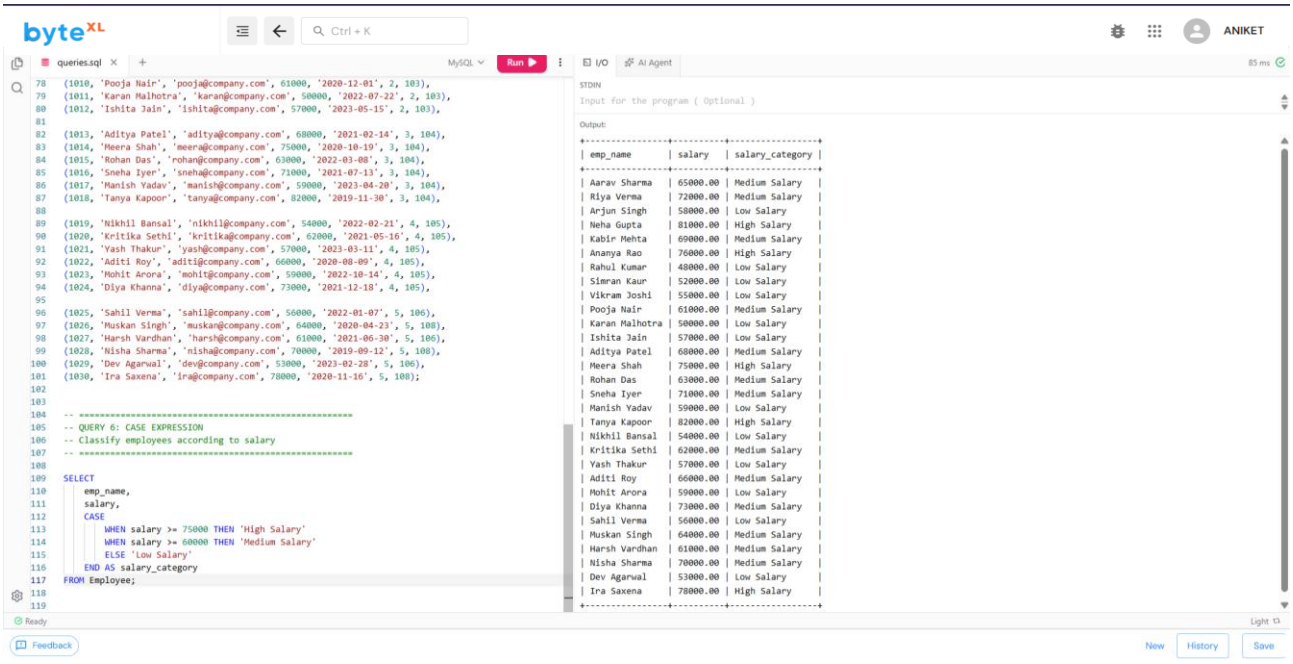

```
-- 4. GROUP BY: department-wise employee count and average salary
SELECT dept_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS average_salary
FROM Employee
GROUP BY dept_id;
```



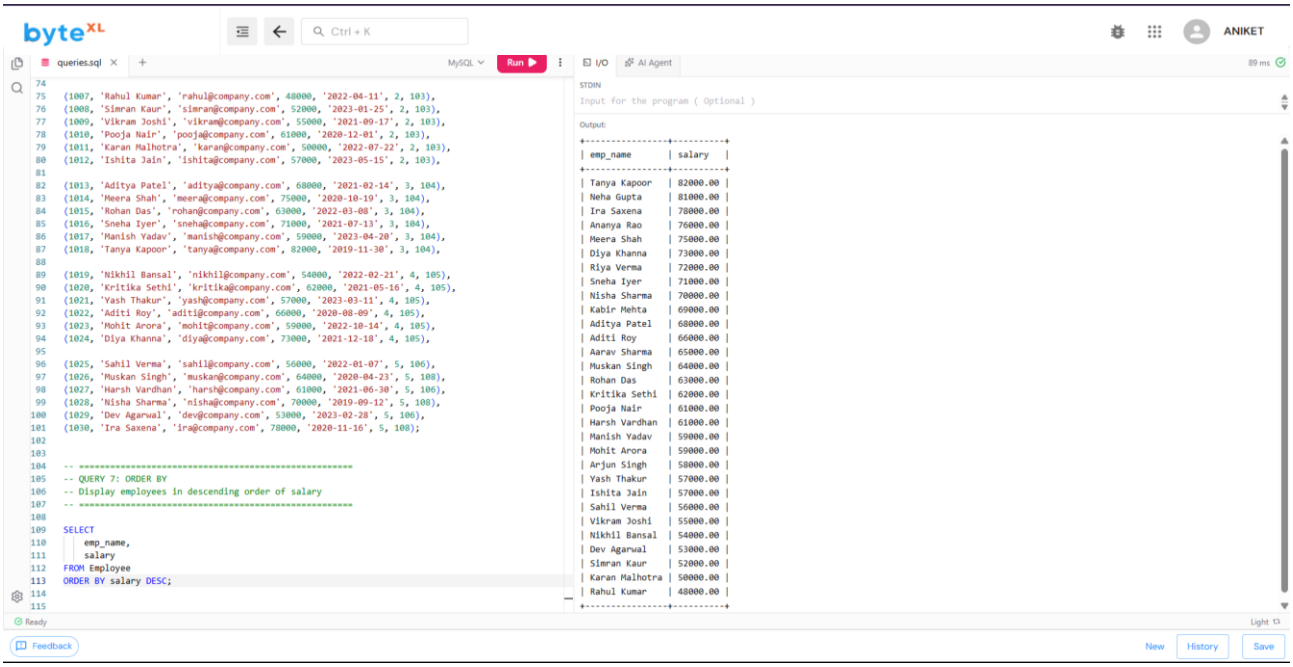

```
-- 5. HAVING: departments having more than 5 employees
SELECT dept_id,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY dept_id
HAVING COUNT(*) > 5;
```



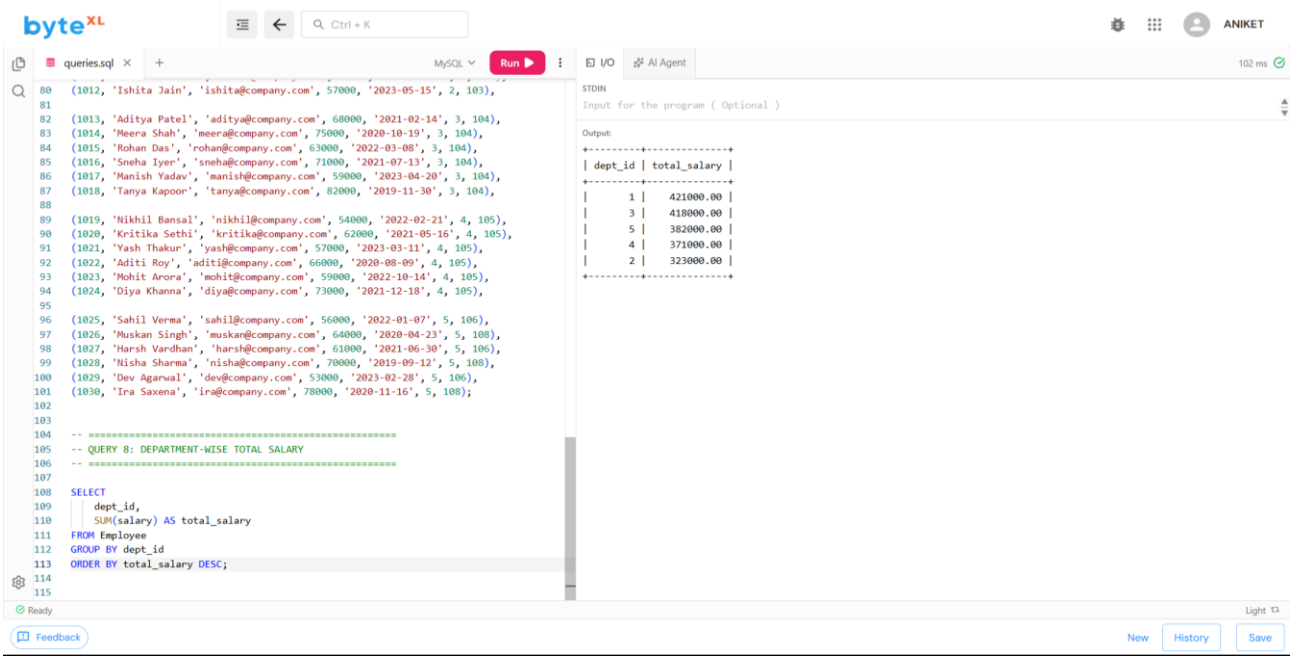

```sql
-- 6. CASE expression: classify employees according to salary
SELECT emp_name,
       salary,
       CASE
            WHEN salary >= 75000 THEN 'High Salary'
            WHEN salary >= 60000 THEN 'Medium Salary'
            ELSE 'Low Salary'
       END AS salary_category
FROM Employee;
```



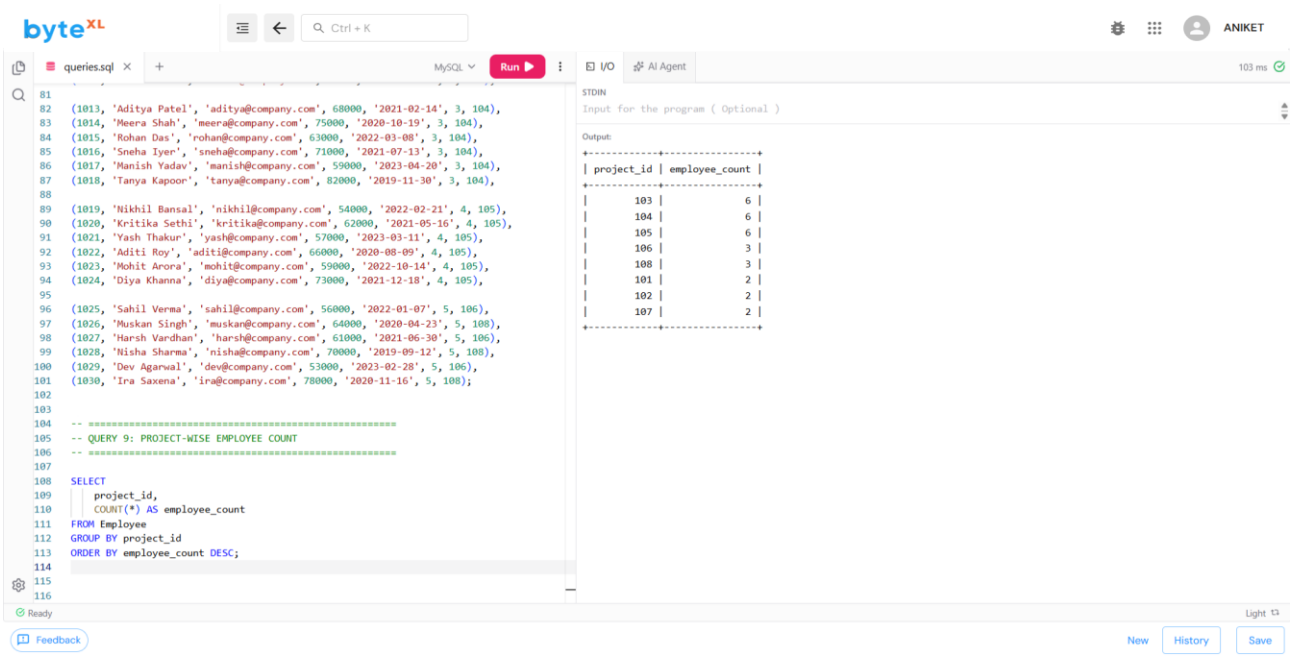

```sql
-- 7. ORDER BY: display employees in descending order of salary
SELECT emp_name, salary
FROM Employee
ORDER BY salary DESC;
```




```
-- 8. Department-wise total salary
SELECT dept_id,
       SUM(salary) AS total_salary
FROM Employee
GROUP BY dept_id
ORDER BY total_salary DESC;
```




```
-- 9. Project-wise employee count
SELECT project_id,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY project_id
ORDER BY employee_count DESC;
```




```sql
-- 10. Display employees with department and project details
SELECT e.emp_name,
       d.dept_name,
       p.project_name,
       e.salary
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
JOIN Project p ON e.project_id = p.project_id
ORDER BY e.salary DESC;
```

